STEP 1 — Create Notebook

STEP 2: Import Required Libraries

```
# ============================
# STEP 2: Import Required Libraries
# ============================

# Data handling
import numpy as np          # For numerical computations
import pandas as pd         # For data manipulation and analysis

# Text preprocessing
import re                   # For regular expressions (text cleaning)
import string               # For punctuation removal

# Visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Machine Learning tools
from sklearn.model_selection import train_test_split   # For splitting dataset
from sklearn.feature_extraction.text import TfidfVectorizer  # TF-IDF vectorization
from sklearn.linear_model import LogisticRegression    # Logistic Regression model
from sklearn.naive_bayes import MultinomialNB          # Naive Bayes model (comparison)
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

STEP 3 — Load and Preprocess Data

```
# ============================
# STEP 3: Load Dataset
# ============================

# Load dataset (Assuming CSV file with columns: review, sentiment)
df = pd.read_csv("/content/IMDB Dataset.csv")

# Display first 5 rows
print(df.head())
```

```
                                              review sentiment
0  One of the other reviewers has mentioned that ...  positive
1  A wonderful little production. <br /><br />The...  positive
2  I thought this was a wonderful way to spend ti...  positive
3  Basically there's a family where a little boy ...  negative
4  Petter Mattei's "Love in the Time of Money" is...  positive
```

```
# ============================
# Text Cleaning Function
# ============================
# Function to clean text
def clean_text(text):
    text = text.lower()  # Convert to lowercase
    text = re.sub(r'\d+', '', text)  # Remove numbers
    text = text.translate(str.maketrans('', '', string.punctuation))  # Remove punctuation
    text = re.sub(r'\s+', ' ', text).strip()  # Remove extra spaces
    return text

# Apply cleaning
df['clean_review'] = df['review'].apply(clean_text)

# Display sample processed text
print(df[['review', 'clean_review']].head())
```

```
                                              review  \
0  One of the other reviewers has mentioned that ...
1  A wonderful little production. <br /><br />The...
2  I thought this was a wonderful way to spend ti...
3  Basically there's a family where a little boy ...
4  Petter Mattei's "Love in the Time of Money" is...

                                        clean_review
0  one of the other reviewers has mentioned that ...
1  a wonderful little production br br the filmin...
2  i thought this was a wonderful way to spend ti...
3  basically theres a family where a little boy j...
4  petter matteis love in the time of money is a ...
```

STEP 4 — Feature Extraction (TF-IDF)

```
# ============================
# STEP 4: TF-IDF Vectorization
# ============================

# Initialize TF-IDF vectorizer
vectorizer = TfidfVectorizer(stop_words='english')

# Convert text to numerical features
X = vectorizer.fit_transform(df['clean_review'])

# Target variable
y = df['sentiment']

# Display vocabulary size
print("Vocabulary Size:", len(vectorizer.vocabulary_))
```

```
Vocabulary Size: 175408
```

STEP 5 — Train Logistic Regression Model

```
# ============================
# STEP 5: Train Logistic Regression
# ============================

# Split dataset into training and testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Initialize Logistic Regression
model = LogisticRegression(max_iter=200)  # Increase max_iter for convergence

# Train model
model.fit(X_train, y_train)

print("Model training completed.")
```

```
Model training completed.
```

STEP 6 — Model Evaluation

```
# ============================
# STEP 6: Evaluation
# ============================

# Make predictions
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

# Classification report
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))

# Confusion matrix
cm = confusion_matrix(y_test, y_pred)

# Plot confusion matrix
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix - Logistic Regression")
plt.show()
```

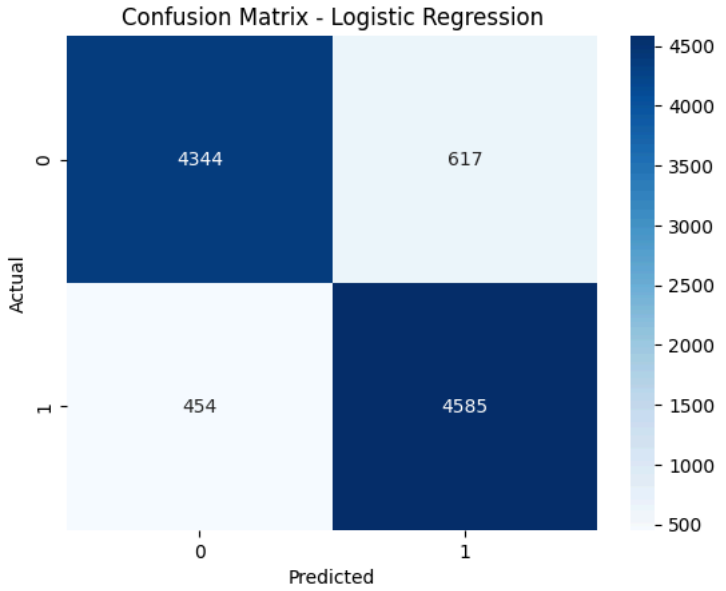
```
Accuracy: 0.8929

Classification Report:

              precision    recall  f1-score   support

    negative       0.91      0.88      0.89      4961
    positive       0.88      0.91      0.90      5039

    accuracy                           0.89     10000
   macro avg       0.89      0.89      0.89     10000
weighted avg       0.89      0.89      0.89     10000
```



STEP 7 — Analysis (Logistic Regression vs Naive Bayes)

- Logistic Regression is a discriminative model, while Naive Bayes is a generative model.
- Logistic Regression directly models the probability $P(y \mid x)$ P(y|x), whereas Naive Bayes models $P(x \mid y)$ P(x|y).
- Naive Bayes assumes feature independence, which is rarely true for text data.
- Logistic Regression does not assume independence, making it more flexible.
- Logistic Regression usually performs better with large datasets.
- Naive Bayes trains faster and works well with small datasets.
- Logistic Regression provides interpretable coefficients.
- Naive Bayes handles high-dimensional sparse data efficiently.
- Logistic Regression may require tuning (max_iter, regularization).
- Overall, Logistic Regression often achieves higher accuracy in sentiment classification tasks.